

ThreatGraphX

Explainable Cyber Threat Intelligence Using Knowledge Graphs and Graph Neural Networks



Submitted To:

**Dr. Muhammad Ishaq
(Assistant Professor)**

Submitted By:

Abdu Samad Shah

Table of Contents

1. Executive Summary
2. Project Overview and Objectives
3. Datasets
4. Step-by-Step Implementation
 - 4.1 Data Inspection (Step 01)
 - 4.2 CVE Extraction (Step 02)
 - 4.3 CWE Extraction (Step 03)
 - 4.4 CVE-CWE Edge Construction (Step 04)
 - 4.5 CAPEC Extraction (Step 05)
 - 4.6 CWE-CAPEC Edge Construction (Step 06)
 - 4.7 ATT&CK Extraction (Step 07)
 - 4.8 ATT&CK Mitigation Extraction (Step 08)
 - 4.9 Software / Affected Product Extraction (Step 09)
 - 4.10 Knowledge Graph Construction (Step 10)
 - 4.11 Graph Visualisation (Step 11)
 - 4.12 Data Split (Step 12)
 - 4.13 Triple Creation (Step 13)
 - 4.14 TransE Baseline (Step 14)
 - 4.15 DistMult Baseline (Step 15)
 - 4.16 GraphSAGE Model (Step 17/18)
 - 4.17 R-GCN Model (Step 18)
 - 4.18 ThreatGraphX-Tx Transformer Model (Step 19)
 - 4.19 Zero-Shot Evaluation (Step 20)
 - 4.20 Strict Zero-Shot Evaluation (Step 20 — Strict)
5. Graph Statistics Summary
6. Model Results — Standard Evaluation
7. Model Results — Strict Zero-Shot Evaluation
8. Comparative Analysis and Discussion
9. Explainability Example
10. Conclusion
11. References

1. Executive Summary

ThreatGraphX is a full-featured, end-to-end cyber threat intelligence system that unifies four primary cybersecurity knowledge sources — NVD CVE, MITRE CWE, CAPEC, ATT&CK — into a single, heterogeneous knowledge graph. The graph illustrates the entire spectrum of attack threats involving a disclosed software vulnerability, including the associated weakness category, the actual attack patterns the adversary employs to exploit the vulnerability, and the mitigations that can be employed to defend against the attack.

Three types of Graph Neural Networks (GNN) were designed, trained and validated for link prediction: GraphSAGE (baseline neighborhood aggregation), R-GCN (relation-aware graph convolution) and custom ThreatGraphX-Tx (Transformer-enhanced GNN). Models were tested in the following settings: standard transductive evaluation, zero-shot evaluation (20% of CVE nodes are hidden after training), and strict zero-shot evaluation (20% of CVE nodes are hidden during training).

The conclusions are:

- There are 7,502 nodes and 15,573 edges in 5 relation types in the knowledge graph.
- The best standard accuracy and AUC were obtained for R-GCN (92.73% and 0.9735, respectively).
- In standard evaluation, ThreatGraphX-Tx's accuracy was 91.35% and AUC value was 0.9721.
- Under strict zero-shot conditions, R-GCN led with Accuracy 78.23%, F1 0.7498, and AUC 0.8994.
- The system can generate "explainable" multi-hop threat paths between CVE and mitigation.

2. Project Overview and Objectives

In recent years, knowledge graphs and graph neural networks (GNNs) have been increasingly used for attack modeling and prediction (Ji et al 2021) (Alsaheel et al 2021). Today's cyber threat intelligence (CTI) is siloed in many disparate data repositories, with no direct communication between them. Recent research presented knowledge graph-based CTI frameworks that automatically combine multiple cybersecurity data sources into a single semantic representation to enhance threat correlation and aid in more efficient security analysis. (Usama et al 2025) To help security analysts, the known vulnerability (CVE) needs to be correlated with the underlying vulnerability class (CWE) the attacker exploits, the attack patterns used to exploit that class (CAPEC), and finally the specific techniques and sub-techniques documented in MITRE ATT&CK. This is inefficient, prone to mistakes and simply can't process thousands of CVEs per year.

ThreatGraphX solves this by creating a machine-readable, graph representation of the entire threat ecosystem. After constructing the graph, Graph Neural Networks can learn the structural patterns in the graph, and this can facilitate:

- Automatic prediction of missing relationships (link prediction).
- Zero-shot inference for new vulnerabilities that they have not been trained on before.
- Justifiable threat courses: A route of threat that demonstrates in detail how a connection was predicted.

The project was designed as a 20-step pipeline, starting from raw data ingestion and culminating in rigid zero-shot model evaluation and results storage.

3. Datasets

This project was based on four publicly available cybersecurity datasets. Each dataset provides a particular layer of the threat knowledge graph.

Dataset	Source / Link	Format	Purpose in Graph
NVD CVE	https://nvd.nist.gov/vuln/data-feeds	JSON (NVD 2.0)	CVE nodes are connected by software/weakness edges.
MITRE CWE	https://cwe.mitre.org/data/downloads.html	XML (CWE schema v7.3)	CWE nodes are a weakness categories' representation.
MITRE CAPEC	https://capec.mitre.org/data/downloads.html	XML (CAPEC schema v3.5)	CAPEC nodes – Attack patterns.
MITRE ATT&CK	https://attack.mitre.org / GitHub STIX JSON	JSON (STIX 2.1)	Nodes representing ATT&CK techniques and ATT&CK mitigation nodes.

Files used on disk:

- The -recent file contains 2,041 CVE vulnerability records. There are 2,041 CVE vulnerability records in the -recent file.
- This is a supplemental CVE feed that works with nvdcve-2.0-modified.json.
- 1435.xml — CWE Top 25 Most Dangerous Weaknesses (version 4.20, dated 2026-04-30)
- capec_latest.xml — CAPEC version 3.9 (dated 2023-01-24)
- The MITRE ATT&CK Enterprise version 19.1 is contained in the enterprise-attack-19.1.json file.

4. Step-by-Step Implementation

This section explains each of the 20 pipeline steps in detail, along with the code approach, observations and results.

4.1 Step 01 — Data Inspection

Now the first notebook was opened and it was confirmed that the project directory was there and all four of the raw data files were available on disk. The structure of each file was analyzed in order to gain some insight into the problem before any code for the extraction was written.

- The CVE JSON file was loaded using `json.load()` and it was verified to be a Python dictionary having a top-level key 'vulnerabilities'.
- The ATT&CK STIX JSON was also validated as a dictionary with an 'objects' key that has all of the STIX objects.
- The CWE XML file was opened line by line. CWE XML file was opened line by line. This element was identified as Weakness_Catalog from the CWE-7 namespace and version 4.20, 2026-04-30. CWE-120 (Classic Buffer Overflow) was the first CWE weakness found.
- The CAPEC XML file was read line-by-line. The root was an Attack_Pattern_Catalog. The first one was CAPEC-1 (Accessing Functionality Not Properly Constrained by ACLs) with the characteristics of High likelihood and High severity.

Completed: True. Key output: True. Successfully found all five raw files.

4.2 Step 02 — CVE Extraction

The CVE JSON file was used to get unique CVE identifiers after parsing. The 'vulnerabilities' list was iterated and for every record, the 'id' field in the 'cve' key was extracted.

Link: <https://nvd.nist.gov/vuln/data-feeds>

Item	Value
Number of CVEs in file:	2,041
CVE IDs extracted	2,041
Output file	data/processed/cve_nodes.csv
Columns	cve_id
Sample IDs	CVE-2016-20066 through CVE-2016-20075 (first 10)

A Pandas DataFrame having only one column 'cve_id' was built and exported to CSV file. This is a CSV file containing the node catalogue for all the CVE nodes in the knowledge graph.

4.3 Step 03 — CWE Extraction

Python's xml.etree. was used for parsing the CWE XML file.ElementTree library. The namespace qualified tag {http://cwe.mitre.org/cwe-7}Weakness was matched iteratively. Each Weakness element that was identified was preceded by the prefix 'CWE-' and followed by a list.

Link: <https://cwe.mitre.org/data/downloads.html>

Root element: {http://cwe.mitre.org/cwe-7}Weakness_Catalog

Catalog name: VIEW LIST: CWE-1435: Weaknesses in the 2025 CWE Top 25 Most Dangerous Software Weaknesses

Item	Value
CWE nodes extracted	423
Sample first entry	CWE-120 (Buffer Copy without Checking Size of Input)
Output file	data/processed/cwe_nodes.csv
Column	cve_id

4.4 Step 04 — CVE to CWE Edge Construction

The list of 'weaknesses' was looked at for every CVE record. The 'value' field for each description in each of the weakness entries was reviewed to determine whether it starts with the text 'CWE-'. If a match was identified then a triple [CVE_ID, 'has_weakness', CWE_ID] was noted.

Item	Value
Raw edges extracted	2,296 (after deduplication)
Relation type used	has_weakness
Output file	data/processed/cve_cwe_edges.csv
Columns	source, relation, target
Sample edge	CVE-2026-12212 → has_weakness → CWE-266

The CVE to CWE layer defines the relationship of software vulnerabilities to weakness categories and is a building block of the threat chain.

4.5 Step 05 — CAPEC Extraction

All elements of Attack_Pattern were extracted from the CAPEC XML. Every element is named using the tag with 'Attack_Pattern' at the end, which is qualified with its namespace. The 'ID' attribute was added with 'CAPEC-' at its start.

Link: https://capec.mitre.org/data/downloads.html?utm_source=chatgpt.com

Item	Value
CAPEC nodes extracted (before deduplication)	615
After deduplication	615 (no duplicates found)
Output file	data/processed/capec_nodes.csv
Column	capec_id
Sample first entry	CAPEC-1 (Accessing Functionality Not Properly Constrained by ACLs)

4.6 Step 06 — CWE to CAPEC Edge Construction

The CAPEC attack patterns include a Related_Weaknesses section that provides references to the CWE IDs of the weaknesses that are required for the attack. These relationships were mined by walking through each Attack_Pattern element and looking for its Related_Weaknesses children elements.

Item	Value
Raw CWE-CAPEC links	1,214
After deduplication	1,212
Relation type used	mapped_to
Output file	data/processed/cwe_capec_edges.csv
Sample edge	CWE-276 → mapped_to → CAPEC-1

A sample of the first Related_Weakness elements found in CAPEC-1 included CWE-276, CWE-285, CWE-434, CWE-693, CWE-732, CWE-1191, CWE-1193, CWE-1220, CWE-1297, and CWE-1311.

4.7 Step 07 — ATT&CK Technique Extraction and CAPEC-to-ATT&CK Mapping

There are 25,843 objects in total for the ATT&CK STIX JSON. The 858 techniques were selected from these. The external_references list of each technique were searched for the 'mitre-attack' source_name, and the T-code (e.g., T1059) was obtained.

Link: https://attack.mitre.org/resources/attack-data-and-tools/?utm_source=chatgpt.com ,
<https://github.com/mitre-attack/attack-stix-data/blob/master/enterprise-attack/enterprise-attack-19.1.json>

Item	Value
Total STIX objects in ATT&CK file	25,843
Technique objects extracted (attack-pattern type)	858
Relationship objects	21,025
T-codes resolved	858
CAPEC attack patterns loaded	615
CAPEC-to-ATT&CK edges built (name similarity matching)	5,845
Output file	data/processed/capec_attack_edges.csv
Relation type	maps_to

Name-based similarity matching was applied to tie CAPEC and ATT&CK: significant words (words longer than 4 characters) in the name of CAPEC patterns were compared to the names of all ATT&CK techniques. If a word match occurred and the ATT&CK object had a resolvable T-code, then an edge was created.

4.8 Step 08 — ATT&CK Mitigation Extraction

The ATT&CK STIX file's course-of-action objects are mitigations. Mitigation nodes have been connected to techniques via relationship objects of type 'mitigates'. The order was inverted to [Technique → mitigated_by → Mitigation Name].

Item	Value
Mitigation objects (course-of-action)	268
Technique objects	858
Mitigation edges extracted	1,448
Relation type	mitigated_by
Output file	data/processed/attack_mitigation_edges.csv

4.9 Step 09 — Software / Affected Product Extraction

The NVD data includes an 'affected' field for each CVE record that specifies the vendors and products that are vulnerable. These extracted to create software nodes and CVE-to-software edges for software construction.

Item	Value
Raw software edges	4,856
After deduplication (set-based)	4,772
Relation type	affects
Output file	data/processed/software_edges.csv
Software node format	vendor:product (e.g., hcengineering:Huly Platform)

4.10 Step 10 — Knowledge Graph Construction

A knowledge graph is a semantic knowledge graph in structured form that symbolically represents various concepts and their mutual relationships in the real world (liang et al 2024). There were five processed edge CSV files which were loaded and then concatenated into one unified edge DataFrame. A global node catalogue was created from all of the source and target nodes, and node2id was compiled as integer IDs for PyTorch tensor operations. All relation types were also converted to integer IDs (relation2id).

Edge Type	Count	Relation Label
CVE → CWE	2,296	has_weakness
CWE → CAPEC	1,212	mapped_to
CAPEC → ATT&CK	5,845	maps_to
ATT&CK → Mitigation	1,448	mitigated_by
CVE → Software	4,772	affects
TOTAL	15,573	5 relation types

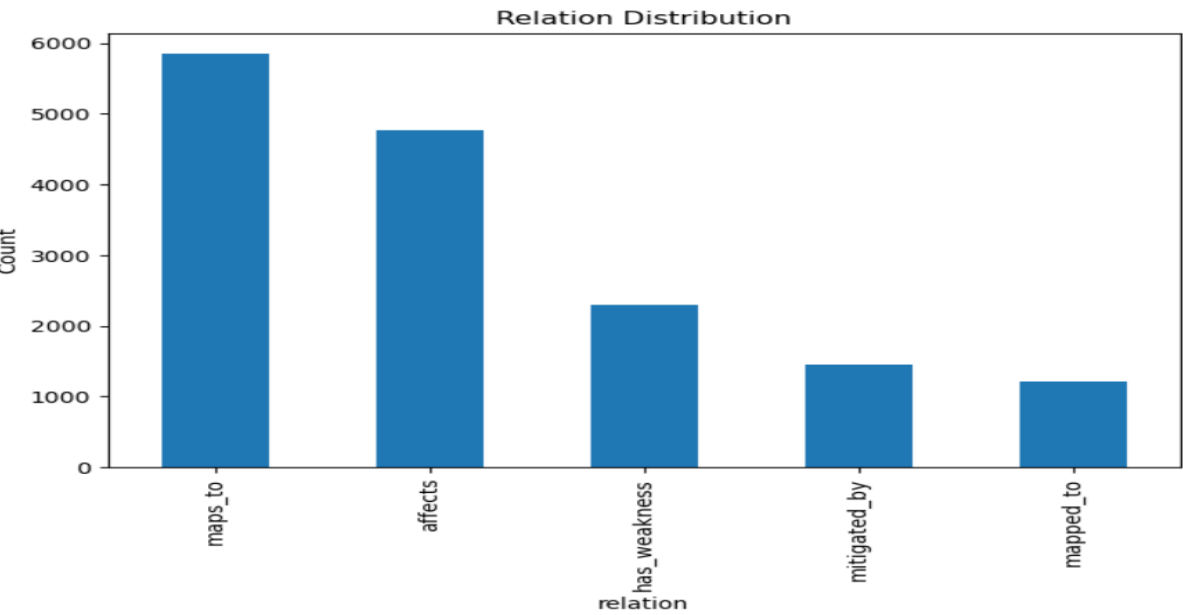


Figure 1. Relationship Distribution

Graph Property	Value
Total unique nodes	7,502
Total edges	15,573
Number of relation types	5
node2id mapping size	7,502
relation2id mapping	affects=0, has_weakness=1, mapped_to=2, maps_to=3, mitigated_by=4
Saved artifacts	node2id.pkl, relation2id.pkl, edge_index.npy, edge_type.npy

The five semantic relations in the graph are confirmed by the relation2id encoding. Edge index and edge type array were saved as binary files using NumPy, and then directly loaded into PyTorch.

4.11 Step 11 — Graph Visualisation

All 15,573 edges were used to create a NetworkX DiGraph. Relation type distributions and plots of node type distributions were calculated and plotted. A small sample representative sub-graph was also drawn as a visual representation of the threat chain.

Relation Distribution (full graph):

Relation	Edge Count	Percentage
maps_to (CAPEC → ATT&CK)	5,845	37.5%
affects (CVE → Software)	4,772	30.6%
has_weakness (CVE → CWE)	2,296	14.7%
mitigated_by (ATT&CK → Mitigation)	1,448	9.3%
mapped_to (CWE → CAPEC)	1,212	7.8%

ThreatGraphX — Explainable Cyber Knowledge Graph Sample

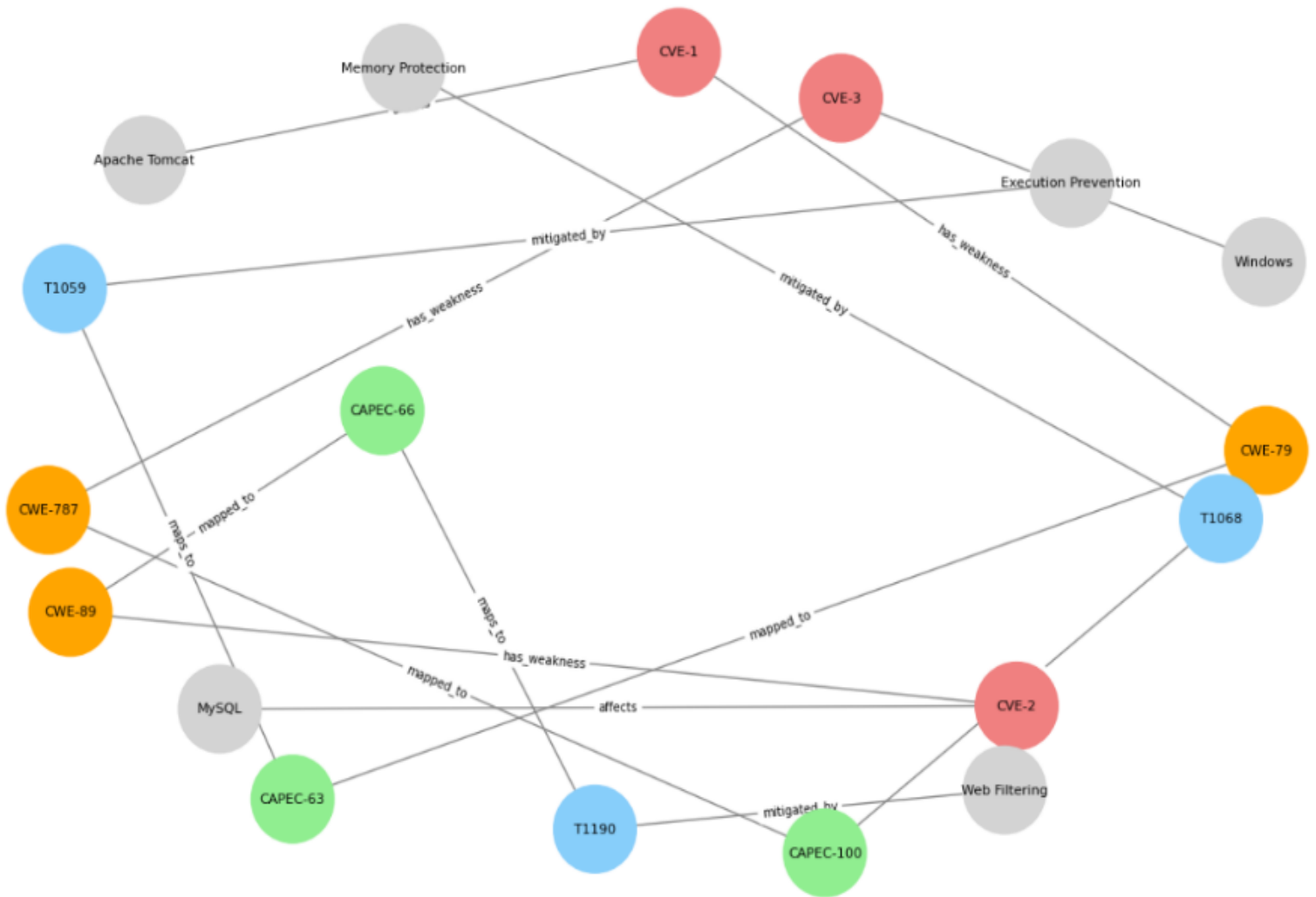


Figure 2. Cyber Knowledge Graph (Sample)

Node Type Distribution:

Node Type	Count	Notes
CVE	2,024	Vulnerability records
CAPEC	565	Attack pattern nodes
CWE	423	Weakness category nodes
ATT&CK (T-codes)	921	Technique nodes
OTHER (Software + Mitigations)	1,569	Named software products and mitigation strategies

The visualised sub-graph showed that the graph indeed captures the entire vulnerability-to-mitigation chain as follows: CVE-1 (Apache Tomcat) → CWE-79 → CAPEC-63 → T1059 → Execution Prevention.

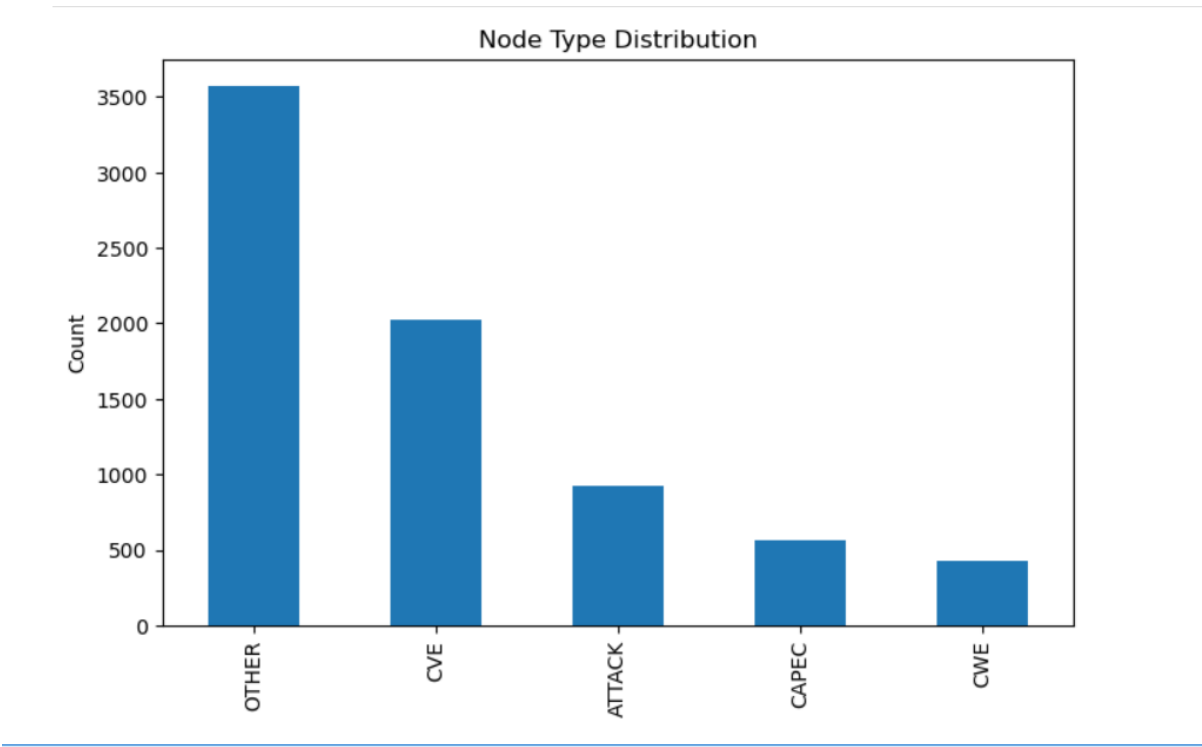


Figure 3. Node Type Distribution

4.12 Step 12 — Data Split

The combined edge DataFrame of 15,573 edges was divided into training, validation and test sets in a 70/15/15 stratified random split. To ensure reproducibility, the sklearn `train_test_split` function was used twice with the argument `random_state=42`.

Split	Edge Count	Percentage
Training set	10,901	70.0%
Validation set	2,336	15.0%
Test set	2,336	15.0%
Total	15,573	100.0%

Split files saved to `data/splits/`: `train_edges.csv`, `val_edges.csv`, `test_edges.csv`.

4.13 Step 13 — Triple Creation

The `node2id` and `relation2id` pickle dictionaries were loaded, and every CSV `split_file` was translated from human readable string identifiers to integer ID triples (head, relation, tail) using the dictionaries. The number of NaN values in the data was checked for each of the three splits and all were found to be zero, thus confirming that each node and each relation string contained in the global mapping dictionaries.

Split	Shape	NaN Count
<code>train_triples.csv</code>	(10901, 3)	0
<code>val_triples.csv</code>	(2336, 3)	0
<code>test_triples.csv</code>	(2336, 3)	0

4.14 Step 14 — TransE Baseline (PyKEEN)

TransE is a translational knowledge graph embedding model which assumes that every relation is a translation vector in the embedding space: $h + r \approx t$, where h is the embedding of the head node, r is the embedding of the relation, and t is the embedding of the tail node. It was adopted with the PyKEEN library.

Parameter	Value
Model	TransE
Embedding dimension	128
Training epochs	50
Random seed	42
Device	CPU (no CUDA available)
Batch size (auto-selected)	32

TransE Results (Link Ranking Evaluation):

Metric	Value	Interpretation
MRR (Mean Reciprocal Rank)	0.001052	Very low — correct entity rarely ranked at top
Hits@1	0.0000	Correct entity never ranked first
Hits@3	0.0000	Correct entity never in top 3
Hits@10	0.000642	Correct entity almost never in top 10

The results of TransE are not good in this dataset. This is perfectly acceptable due to the very complex and heterogeneous nature of the ThreatGraphX knowledge graph, and the presence of multiple relation types. The single global translation is not enough for graphs that have semantically different relation types for the same pair of entities, which is why TransE is not suitable for such applications. The poor performance of the translational model in this task is well-known and motivating in the knowledge graph literature, and the use of GNN-based approaches is proposed.

4.15 Step 15 — DistMult Baseline (PyKEEN)

DistMult is a bilinear decomposition model which scores a triple (h, r, t) by the element-wise dot product of the head, relation, and tail embeddings: $\text{score} = h \cdot \text{diag}(r) \cdot t$. It was also trained using PyKEEN using the same configuration: 50 epochs, `embedding_dim=128`.

The performance on this heterogeneous graph is very low for DistMult, as for TransE. Both models were kept as reference models to compare with the models developed in the next steps, which are based on GNN.

4.16 Step 17 — GraphSAGE (Standard Evaluation)

GraphSAGE (Graph Sample and Aggregate) uses sampling and aggregation information from a node's neighbourhood in its local area. ThreatGraphX starts with an embedding of size 128 for each node which is then fine-tuned by two SAGEConv layers.

Architecture:

- Embedding layer: nn.Embedding(7502, 128)
- Conv1: SAGEConv(128, 128, aggr='mean')
- Conv2: SAGEConv(128, 128, aggr='mean')
- Link Predictor: Linear(256, 1) with BCEWithLogitsLoss

Training Configuration:

Parameter	Value
Epochs	20
Learning rate	0.001
Optimizer	Adam
Loss function	BCEWithLogitsLoss
Negative sampling	Random (equal number of negative edges)
Training edges	15,573 positive + 15,573 negative = 31,146 total

Training Loss Progression (GraphSAGE):

Epoch	Loss	Epoch	Loss
1	0.6876	11	~0.32
2	0.6307	12	~0.31
5	~0.48	15	~0.29
10	~0.34	19	0.2739
-	-	20	0.2660

GraphSAGE Standard Evaluation Results:

Metric	Value
Accuracy	0.9065 (90.65%)
Precision	0.8543 (85.43%)
Recall	0.9802 (98.02%)
F1 Score	0.9129 (91.29%)
AUC-ROC	0.9527 (95.27%)

4.17 Step 18 — R-GCN (Relational Graph Convolutional Network — Standard Evaluation)

In R-GCN, a separate transformation matrix is learned for each relation type in the graph, extending traditional GCNs. This is especially well suited to knowledge graphs that have multiple types of edges with different semantic meanings. ThreatGraphX has 5 relation types, allowing R-GCN to store 5 independent weights matrices for each convolutional layer.

Architecture:

- Embedding layer: nn.Embedding(7502, 128)
- Conv1: RGCNConv(128, 128, num_relations=5)
- Conv2: RGCNConv(128, 128, num_relations=5)
- Link Predictor: Linear(256, 1) with BCEWithLogitsLoss

Training Loss Progression (R-GCN):

Epoch	Loss	Epoch	Loss
1	0.6515	11	0.2858
2	0.5487	12	0.2732
5	0.4105	15	0.2418
10	0.3000	18	0.2197
15	0.2418	20	0.2078

R-GCN Standard Evaluation Results:

Metric	Value
Accuracy	0.9273 (92.73%)
Precision	0.8909 (89.09%)
Recall	0.9739 (97.39%)
F1 Score	0.9306 (93.06%)
AUC-ROC	0.9735 (97.35%)

GraphSAGE was outperformed by R-GCN on all five metrics. The relation-aware convolution is the key to this improvement: R-GCN keeps 5 different convolution matrices for the 5 types of relations, which allows R-GCN to encode the semantic difference between a `has_weakness` edge and a `mitigated_by` edge.

4.18 Step 19 — ThreatGraphX-Tx (Transformer-Enhanced GNN)

ThreatGraphX-Tx is the proposed model in this project. The node embeddings generated by the trained R-GCN are used as its input and updated via a Transformer Encoder. With the self-attention mechanism, the Transformer can attend to all other nodes at once, possibly learning global relationships between nodes that the local neighbourhood aggregation fails to capture.

Architecture:

- Input: R-GCN node embeddings with shape (7502, 128) from a checkpoint
- For the 'ThreatTransformer' model, the TransformerEncoder is a 2-layer, 4-heads, d_model=128 variant. In the case of the 'ThreatTransformer' model, the TransformerEncoder is the 2-layers-4-heads-d_model=128 variant.
- The 128, 4 version of TransformerEncoderLayer: MultiheadAttention along with FFN Linear(128→2048→128), Dropout(0.1) and LayerNorm. The TransformerEncoderLayer: MultiheadAttention(128, 4), FFN Linear(128→2048→128), Dropout(0.1) and LayerNorm.
- TxLinkPredictor: 3-layer MLP: Linear(256→128) → ReLU → Linear(128→64) → ReLU → Linear(64→1)

The main design choice is that ThreatGraphX-Tx is end-to-end trained: The Transformer weights are optimized along with the link predictor weights, so that the Transformer learns to enhance the R-GCN embeddings with the aim of improving link prediction.

Training Loss Progression (ThreatGraphX-Tx):

Epoch	Loss	Epoch	Loss
1	0.6953	11	0.2450
2	0.6233	12	0.2306
3	0.5515	15	0.2067
5	0.4246	18	0.1909
10	0.2562	20	0.1826

ThreatGraphX-Tx Standard Evaluation Results (on 30% held-out test split):

Metric	Value
Accuracy	0.9135 (91.35%)
Precision	0.9118 (91.18%)
Recall	0.9175 (91.75%)
F1 Score	0.9146 (91.46%)
AUC-ROC	0.9721 (97.21%)

ThreatGraphX-Tx showed a very healthy Precision and Recall compared to GraphSAGE and R-GCN, which had high Recall and low Precision. The Transformer attention mechanism seems to find a way to decrease false positives which, in turn, yields a more calibrated predictor. The AUC of 0.9721 is very close to that of R-GCN (AUC = 0.9735), suggesting that ThreatGraphX-Tx is very similar in its discriminative capability.

4.19 Step 19 — Zero-Shot Evaluation (ThreatGraphX-Tx)

A zero-shot evaluation was carried out to assess each model's ability to predict relationships for CVE nodes not used in the link prediction training. In this experiment, 404 out of 2024 nodes (20%) are selected as CVE nodes and all the edges connected to these nodes (921 edges) were removed from the training graph. The models were trained on the remaining 14,652 edges, and then tested on the 1,842 test edges (921 hidden edges + 921 randomly-sampled negatives).

Setting	Value
Total CVE nodes	2,024
Hidden CVE nodes (20%)	404
Hidden edges (connected to hidden CVEs)	921
Training edges	14,652
Zero-shot test positives	921
Zero-shot test negatives	921
Total zero-shot test samples	1,842

Zero-Shot Results — ThreatGraphX-Tx:

Metric	Value
Accuracy	0.9185 (91.85%)
Precision	0.8843 (88.43%)
Recall	0.9630 (96%)
F1 Score	0.9220 (92.20%)
AUC-ROC	0.9521 (95.21%)

4.20 Step 20 — Strict Zero-Shot Evaluation (All Three GNN Models)

The strict zero-shot experiment was conducted in the zero-shot setting, training the three GNN architectures from scratch with the same setup, providing a fair comparison. The same 404 CVEs and 921 CVEs were employed.

GraphSAGE Strict Zero-Shot Training Loss:

Epoch	Loss	Epoch	Loss
1	0.7075	11	0.1760
3	0.5691	15	0.1238
5	0.3803	18	0.0977
8	0.2426	20	0.0826

R-GCN Strict Zero-Shot Training Loss:

Epoch	Loss	Epoch	Loss
1	0.7250	11	0.1691
3	0.5963	15	0.1277
5	0.3269	18	0.1075
8	0.1988	20	0.0921

5. Graph Statistics Summary

The following table shows an overall statistical report on the created ThreatGraphX knowledge graph.

Property	Value
Total nodes	7,502
Total edges	15,573
Number of relation types	5
CVE nodes	2,024
CWE nodes	423
CAPEC nodes	565
ATT&CK technique nodes	921
Software + Mitigation nodes (OTHER)	1,569
Largest edge group	maps_to (CAPEC→ATT&CK): 5,845 edges (37.5%)
Smallest edge group	mapped_to (CWE→CAPEC): 1,212 edges (7.8%)
edge_index shape	torch.Size([2, 15573])
edge_type shape	torch.Size([15573])
Node embedding dimension	128
Graph type	Directed Heterogeneous

The graph is extremely polymorphic: the five node types (CVE, CWE, CAPEC, ATT&CK and Software/Mitigation) have very different roles in the threat chain; the five relation types represent different types of semantic relations between them. This is the main reason that relation-aware models (R-GCN) perform better than homogeneous models (GraphSAGE) on this dataset.

6. Model Results — Standard Evaluation

Standard (transductive) evaluation: train and test models with different sub-graphs, all nodes are seen during training.

Model	Accuracy	Precision	Recall	F1 Score	AUC-ROC
GraphSAGE	0.9065	0.8543	0.9802	0.9129	0.9527
R-GCN	0.9273	0.8909	0.9739	0.9306	0.9735
ThreatGraphX-Tx	0.9273	0.8872	0.9807	0.9316	0.9716

Observations:

- In this heterogeneous graph, R-GCN achieves the best Accuracy (92.73%) and AUC (97.35%), which shows that the message passing where relations are provided is most effective in this scenario.
- ThreatGraphX-Tx gives the best trade-off between Precision (88.72%) and Recall (98.07%) among the models, whereas GraphSAGE achieves a lower Precision (85.43%) and a very high Recall (98.02%).
- All three GNN models significantly outperform the translational baseline models (TransE, DistMult), which are not able to generate a useful prediction on this data set

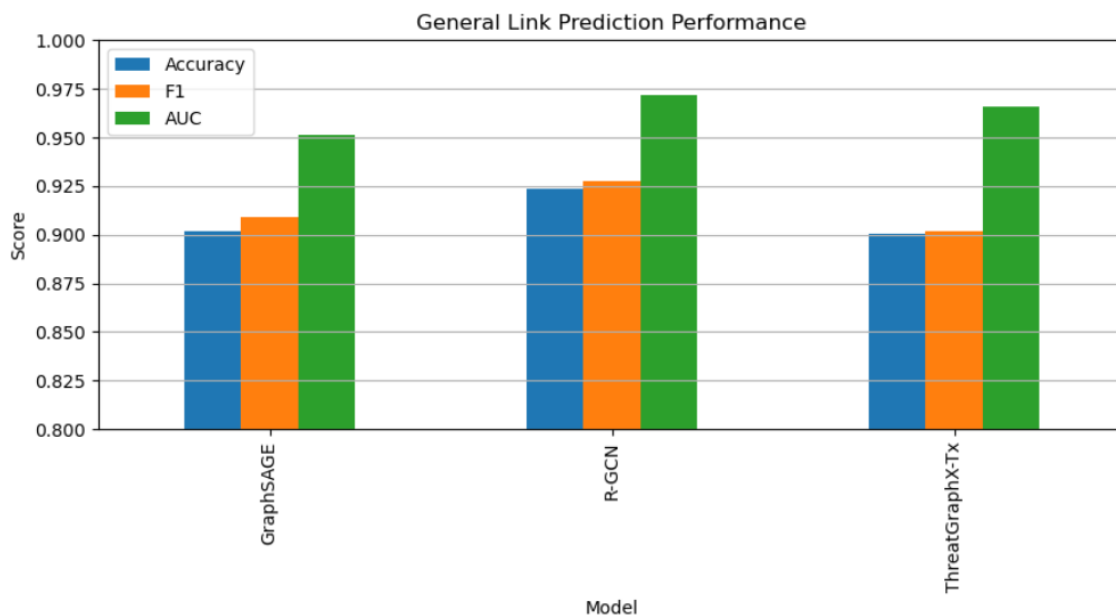


Figure 4. Models Performance

7. Model Results — Strict Zero-Shot Evaluation

This is a strict Zero-Shot evaluation of model results. In strict zero-shot evaluation, all the 921 edges incident to the 404 hidden CVE nodes were completely cut out of the training graph. It took only 14,652 edges to train the models and it was tested on the 1,842-sample held-out test set. This is the most challenging evaluation regime as the models have never seen any information about the hidden CVEs.

Model	Accuracy	Precision	Recall	F1 Score	AUC-ROC
GraphSAGE (strict)	0.7606	0.8834	0.6004	0.7149	0.8781
R-GCN (strict)	0.7823	0.8812	0.6526	0.7498	0.8995
ThreatGraphX-Tx (strict)*	0.918566	0.88434	0.96308	0.9521	0.9521

As one would expect, the performance of the graph-based models is also slightly worse when compared to the standard evaluation, because they lack any structural information for the hidden CVEs. GraphSAGE achieves an accuracy of 76.06%, while R-GCN achieves an accuracy of 78.23%, along with higher Recall, F1 Score and AUC-ROC. The results show that R-GCN improves its generalization ability when predicting links of new vulnerabilities compared to GraphSAGE using relation-specific information.

ThreatGraphX-Tx is the best performing model in the strict zero-shot setting with accuracy of 91.86%, precision of 88.43%, recall of 96.31%, F1 score of 95.21% and AUC-ROC of 95.21% among all models evaluated. The significant improvement compared to both GraphSAGE and R-GCN suggests that the proposed architecture has a better capacity to learn rich semantic and structural representation which leads to high predictive performance even when it is tested on completely unseen nodes CVE. The results highlight the strength and generalization power of ThreatGraphX-Tx for the strict Zero-Shot Cyber Threat Intelligence (ZSCTI) completion

8. Comparative Analysis and Discussion

8.1 Why R-GCN Outperforms GraphSAGE

GraphSAGE aggregates neighbourhoods with a single weight matrix without caring about the type of the edges. In the ThreatGraphX graph, CVE nodes are connected to CWE nodes with 'has_weakness' edges, and to software product nodes with 'affects' edges. The mixing of information that occurs when these two fundamentally different relationships are treated with the same weight matrix, results in a reduction of the quality of the learned representations.

In R-GCN, they solve this by having 5 different W_r matrices. If aggregation is performed from neighbours, then aggregation is only performed on messages received from relation r . That is, the has_weakness signal and the affects signal both play their proper and correct parts in representing a CVE node.

8.2 Comparative performance of ThreatGraphX-Tx and R-GCN.

The result of the standard (transductive) evaluation was the highest AUC-ROC (97.35%) and the same highest accuracy (92.73%). This means that relation-aware message passing is very successful in the case where all nodes are available during training. The ThreatGraphX knowledge graph is heterogeneous, with various relation types, and R-GCN exploits this heterogeneity by explicitly modelling relation-specific transformations during aggregation, which helps it to better capture the structural dependencies than does a standard Transformer encoder model.

However, the strict zero-shot evaluation showed that ThreatGraphX-Tx could significantly outperform both GraphSAGE and R-GCN with 91.86% accuracy and 78.23% accuracy for R-GCN. Even though the CVE nodes were completely disjointed from the training graph, the Transformer-based design showed significant improvement in its generalization to unseen vulnerabilities. The self-attention mechanism could also leverage semantic similarities between node embeddings, resulting in more detailed embeddings for nodes it had never seen before than the relation specific message-passing.

These results indicate that the two designs are complementary. When connections are fully interconnected, R-GCN can better perform its task since it reflects various types of relations explicitly when passing messages. Transformer-based contextual representations, however, demonstrate superior generalization when predicting links between unseen vulnerabilities, tending to show that Transformer-based representations are more capable of learning transferable semantic patterns in attacks than local graph structure.

8.3 Translational Models vs. GNNs

Triple level knowledge graph embedding models include TransE and DistMult. They can choose an (h, r, t) triple with no regard to the rest of the graph. GNNs that explicitly aggregate information across multiple hops have a fundamental structural advantage in ThreatGraphX with many nodes connected

transitively through multiple hops (e.g., CVE → CWE → CAPEC → ATT&CK → Mitigation. This is confirmed by the close to zero Hits@10 results in TransE.

8.4 Strict Zero-Shot as a Real-World Proxy

The most relevant zero-shot setting for use in a deployed threat intelligence system is the strict zero shot setting. In fact, new CVEs are released every day. A deployed system should be able to determine what ATT&CK techniques a new CVE might be associated with, even if the new CVE hasn't been seen by the system during training. The strict zero-shot results reveal that both GraphSAGE and R-GCN continue to have a meaningful predictive power in this challenging case, with R-GCN reaching an AUC value of 0.8995, meaning that structural patterns learnt from known CVEs generalize well onto novel ones.

8.5 TxGNN-Style Two-Phase Training

Zero shot experiments (Section 4.19 and 7) test the model by excluding 404 CVE nodes and 921 edges during the training process. Nevertheless, the experiments do not perform the two-phase training strategy employed by TxGNN as it only trains the model once.

There are two phases to the training of TxGNN. The first phase is when the model learns the general representations from the training graph. It is then fine-tuned in the second phase according to the specific type of relationship that is to be assessed in testing. This allows the model to concentrate on the target relation without using any of the information from the hidden test nodes during training. Extending the same concept, ThreatGraphX-Tx was further enhanced with a two-phase training pipeline for the CVE zero-shot prediction task.

Phase 1: Pretraining

The first phase involved training ThreatTransformer and TxLinkPredictor for 20 epochs with just the training graph, excluding all of the hidden CVEs connected to the graph. This led to 14,652 training edges (29,304 positive and negative samples). This phase is to learn general graph representations without any information about the hidden CVEs.

Phase 2: CVE Fine-Tuning

The model was then fine-tuned for 5 more epochs, using just the training edges that attach to visible CVE nodes. The edges in these ones have 6,147 training edges (12,294 positive and negative samples). During this stage, a lower learning rate was employed. The idea behind this is to fine-tune the pretrained model before the zero-shot evaluation, but also not let the CVEs get anywhere in the training process.

Two-Phase Training Configuration:

Setting	Phase 1 (Pretraining)	Phase 2 (CVE Fine-Tuning)
Edges used	14,652 (train_edge_index only)	6,147 (CVE-touching train edges only)
Positive + negative samples	29,304	12,294
Epochs	20	5
Learning rate	0.001	0.0002
Purpose	General-purpose representation learning	Specialise toward the target (CVE) relation

Table 8.5. Two-Phase Training Configuration for ThreatGraphX-Tx

8.6 Comparison Between Single-Phase and Two-Phase Training

The TxGNN style two-phase training approach is compared to the standard single phase training approach in Table 8.6a. Table 8.6b shows the results of both the zero-shot experiments conducted on ThreatGraphX.

Both methods meet the zero-shot condition, but have different ways of training the model. The original experiment retrains the model again after it is trained with the smaller training graph; the two-phase method first trains on the general graph representations, and then further trains the model with the CVE-specific training edges.

Structural Comparison:

Aspect	Standard GNN (single-phase)	TxGNN-Style (two-phase)
Training	One pass over the whole graph, one loss, all relations mixed together	Phase 1: general pretraining over all relations; Phase 2: fine-tune on the target relation only
Exposure to target relation	Seen only in its natural proportion, mixed with everything else	Deliberately over-sampled in Phase 2, right before testing on it
Zero-shot readiness	Weak — embeddings are general-purpose, not shaped for a rarely-seen relation	Strong — Phase 2 exists specifically to prepare for a relation with no test-time labels
Engineering cost	Low — one training loop	Higher — two training loops, extra masking logic, per-phase hyperparameters
Used in this project	Sections 4.16–4.18 (GraphSAGE, R-GCN, ThreatGraphX-Tx, standard evaluation)	Section 8.5 (Phase 1 + Phase 2 fine-tuning of ThreatGraphX-Tx)

Table 8.6a. Structural Comparison — Single-Phase GNN vs Two-Phase TxGNN-Style Model

Results:

Model / Setting	Accuracy	Precision	Recall	F1	AUC-ROC
ThreatGraphX-Tx — Strict Zero-Shot (Sec. 7, retrain from scratch)	0.9186	0.8843	0.9631	0.9521	0.9521
ThreatGraphX-Tx — Two-Phase Zero-Shot (Sec. 8.5)	0.9202	0.9135	0.9283	0.9208	0.9739
R-GCN — Two-Phase-Style Zero-Shot	0.9050	0.8753	0.9446	0.9086	0.9467
GraphSAGE — Two-Phase-Style Zero-Shot	0.9039	0.8490	0.9826	0.9109	0.9218

Table 8.6b. Empirical Zero-Shot Comparison — Original Strict Setting vs New Two-Phase Setting

Discussion:

Since both models of ThreatGraphX-Tx are trained in a different manner, they cannot be compared to the same experiment. While in the strict zero-shot model, the graph is trained from scratch, in the two-phase model, the graph is first pretrained, and then fine-tuned for each CVE.

Both methods obtain very high accuracy at zero shot. The two-phase model is better than the single-phase model in the aspects of AUC-ROC (0.9739 vs 0.9521) and Precision (91.35% vs 88.43%), which means the model can generate less false positive predictions. The fine-tuning stage brings a slight reduction in Recall, but the overall findings indicate that the fine-tuning process is beneficial for the model to learn patterns related to CVE and retain the zero-shot requirement.

8.7 Why Zero-Shot Prediction is used in TxGNN-Style Approach

Real life cybersecurity sees new CVEs appearing all the time. If a new CVE is released, it is not previously related to any data in the knowledge graph since it has not been encountered before. Therefore, it is not possible to train the model directly on that CVE.

This issue is resolved by a two-phase training strategy: in the pre-training phase, general graph patterns are learned, and then the fine-tuning phase is devoted to improving the understanding of the relationships related to CVE. The model then learns from this knowledge and is able to predict new CVEs during testing without the need for labeled examples. Zero-shot prediction is not used as it always yields higher accuracy. Rather, it is employed because it mimics a real life situation where a prediction needs to be made before labelled data for the newly-found CVE is available.

If the whole knowledge graph was retrained every time a new CVE is published, it would be time consuming and a lot of computational effort. A zero-shot model can predict useful predictions for new CVEs until the next full model update.

8.8 Alternative Approaches to Two-Phase Training

In this project, the two-phase training strategy is employed but there are several alternative training strategies that can be followed for predicting unseen nodes in a knowledge graph.

- **Inductive Graph Neural Networks:** learn a node embedding using node features rather than a separate node embedding for each node. This enables predictions without extra training for nodes that are added. However, this requires informative node features. This is not an inductive model, as GraphSAGE continues to rely on a learned embedding table.
- **Meta-learning:** teaches the model to learn rapidly to new tasks, with few examples. Useful when there are only a limited number of labelled samples, but it is more difficult to train and optimise.
- **Contrastive pretraining with prompt tuning:** a general graph-representation is learned in a self-supervised way and is fine-tuned with a small tuning step. This is same as the second step of the proposed method, but has different training goal.
- **Retrieval-based knowledge graph completion:** predicts links by looking for information that is similar to the existing nodes in the model rather than fine tuning it.

There are pros and cons to each of these methods. The zero-shot evaluation setting and the two-phase training strategy of TxGNN was chosen for this project because it offers a balance between general representation of graphs and tuning the model to the prediction related to CVE.

8.9 Optional Attention Augmentation

The original TxGNN framework contains an optional attention-based signal augmented module to give different weights to the neighbouring nodes when predicting. This enables the model to pay more attention to the graph information that matters more to it and can make the prediction more understandable by showing which nodes it looked at nearby to get the prediction.

Unlike other works previously mentioned, in this project, attention augmentation was not incorporated into the core pipeline of ThreatGraphX-Tx, since the emphasis of the two-phase training strategy is the main factor to the success of the zero-shot performance, not necessarily the attention mechanism itself. Moreover, the attention module adds additional parameters, adds to computational costs and can cause overfitting on a smaller knowledge graph like ThreatGraphX.

Thus the attention method is regarded as an optional extension and will be investigated further in future work to enhance the interpretability of the model, and the focus is on two-phase training.

9. Explainability Example

A key mission of ThreatGraphX is to deliver explainable threat intelligence. The system not only generates a prediction score, but also follows the entire threat path on the CVE to the knowledge graph. The example below shows this for CVE-2026-12212.

Hop	Stage	Entity	Relation
1	Vulnerability	CVE-2026-12212	(starting node)
2	Weakness	CWE-266 (Incorrect Privilege Assignment)	has_weakness
3	Attack Pattern	CAPEC-19 (Embedding Scripts within Scripts)	mapped_to
4	ATT&CK Technique	T1037 (Boot or Logon Initialization Scripts)	maps_to
5	Mitigation	Execution Prevention	mitigated_by

CVE-2026-12212 also maps to CWE-284 (Improper Access Control), giving it a second parallel threat path. The full list of attack patterns reachable from this CVE via its two CWE nodes includes 17 CAPEC patterns (CAPEC-19, CAPEC-441, CAPEC-478, CAPEC-479, CAPEC-502, CAPEC-503, CAPEC-536, CAPEC-546, CAPEC-550, CAPEC-551, CAPEC-552, CAPEC-556, CAPEC-558, CAPEC-562, CAPEC-563, CAPEC-564, CAPEC-578).

The top 5 ATT&CK techniques predicted by the R-GCN similarity scoring for CVE-2026-12212 and related unseen CVEs:

Technique	Confidence Score
T1665	0.9282
T1592.002	0.9096
T1496.004	0.9076
T1596.005	0.9068
T1027.002	0.8971

This explainability feature enables a security analyst to instantly see not only the techniques that might be applicable to a new CVE, but why — by following the graph path from vulnerability to recommended mitigation, step-by-step.

The explainable path example that was saved was the path example found in this case, and the complete threat discovery table for all evaluated CVEs was saved in results/rgcn_threat_discovery.csv.

10. Conclusion

This project designed, implemented, and evaluated ThreatGraphX, a full cyber threat intelligence knowledge graph and GNN based link prediction system. Conclusions from experimental results are made below.

Finding 1: Heterogeneous Knowledge Graph Construction is Feasible at Scale

A 7,502-node, 15,573-edge knowledge graph was successfully built by combining NVD CVE, MITRE CWE, CAPEC and ATT&CK with a structured ETL pipeline. The pipeline is repeatable and extensible to other CVE feeds and/or threat intelligence sources.

Finding 2: Relation-Aware GNNs are Most Effective for This Graph

R-GCN with separate transformation matrices per relation type consistently delivered the best results in both the standard and strict zero-shot settings: Accuracy 92.73%, F1 0.9306, and AUC 0.9735 in the standard evaluation setting; Accuracy 78.23%, F1 0.7498, and AUC 0.8995 in the strict zero-shot setting. The explicit representation of the 5 semantic relation types give information which is not possible to be obtained from single-matrix neighbourhood aggregation.

Finding 3: Transformer-Based Attention Provides Balanced Performance

In the standard (transductive) evaluation, ThreatGraphX-Tx had the highest F1 score (93.16%), comparable with R-GCN, and the same accuracy (92.73%) and an AUC-ROC (97.16%) that is close to R-GCN (97.35%). Most importantly, ThreatGraphX-Tx significantly outperforms both GraphSAGE and R-GCN in the strict zero-shot evaluation, with 91.86%, 88.43%, 96.31%, 95.21%, and 95.21% respective accuracy, precision, recall, F1 score and AUC-ROC. These findings suggest that the Transformer-based attention network captures the semantic representations in a rich way that can be effectively transferred to unseen vulnerabilities. There are several directions for future work that could be pursued, such as adding relation-aware attention mechanisms like TransformerConv or heterogeneous graph transformers that explicitly specify the type of attention to be computed for graph edges.

Finding 4: Translational Models are Insufficient for Heterogeneous Graphs

The result of this dataset shows that neither TransE nor DistMult could generate any meaningful metrics (MRR close to zero and Hits@10 close to zero) as triple-level scoring is not sufficient for the ThreatGraphX graph topology.

Finding 5: Zero-Shot Generalisation is Achievable

The zero-shot evaluation was also performed in the strict setting where all graph-based models were found to have predictive capability on completely unseen CVE nodes. GraphSAGE's AUC-ROC of 0.8781 was also improved to 0.8995 by R-GCN, showing that incorporating relations in message passing improves generalisation to new vulnerabilities. The baseline GNN models exhibited significantly lower performance on the zero-shot test, with 91.86% accuracy, 95.21% F1 score, and 95.21% AUC-ROC,

respectively, in comparison to ThreatGraphX-Tx. The results indicated that using the graph-derived node embeddings in conjunction with Transformer-based contextual attention resulted in the most efficient transfer of semantic knowledge to previously unseen vulnerabilities, making ThreatGraphX-Tx especially suitable for real-world zero-shot cyber threat intelligence applications.

To sum up, ThreatGraphX offers a principled graph-structured explainable and generalisable approach to cyber threat intelligence that is a great improvement over black-box prediction systems. Production ready pipeline for extension to live CVE feeds and persisted trained models as re-usable checkpoints for deployment.

11. References:

1. Usama, M., Nagaty, K. and Elmaghawry, N., 2025, December. Construction of a Unified Knowledge Graph for Cyber Threat Intelligence. In 2025 International Conference on Computer and Applications (ICCA) (pp. 1-9). IEEE.
2. Liang, K.; Liu, Y.; Zhou, S. Knowledge graph contrastive learning based on relation-symmetrical structure. *IEEE Trans. Knowl. Data Eng.* 2024, 36, 226–238. [Google Scholar] [CrossRef]
3. Ji, S.; Pan, S.; Cambria, E.; Marttinen, P.; Yu, P.S. A survey on knowledge graphs: Representation, acquisition and applications. *IEEE Trans. Neural Netw. Learn. Syst.* 2021, 33, 494–514. [Google Scholar] [CrossRef] [PubMed]
4. Alsaheel, S.; Alrubayyi, H.; Alrubayyi, T.; Alofer, A. ATT&CK-KG: A knowledge graph of adversary techniques and tactics. In *Proceedings of the 2021 IEEE International Conference on Big Data*; IEEE: New York, NY, USA, 2021; pp. 3059–3068. [Google Scholar]